

Frontend (React) & Backend (Spring Boot) API Integration Complete Interview Guide

Simple, Clear, Deep Explanations — Full Implementations,
Real-Time Scenarios, Common Issues, Fixes & Interview Q&A

Table of Contents

1. How Frontend & Backend Talk (Big Picture)
2. Request-Response Flow (Visual)
3. REST API Basics (Methods, Status Codes)
4. Spring Boot: Building the API (Layered Architecture)
5. React Basics Recap (Components, State, Props, Hooks)
6. React: Calling the API (fetch vs axios, Deep Dive)
7. Axios Interceptors & Centralized API Client
8. Data Fetching Patterns (useEffect, React Query/SWR)
9. CORS — Why It Breaks & How to Fix (Deep Dive)
10. Authentication Flow Between React & Spring Boot (JWT, Full Implementation)
11. Protecting React Routes (Private Routes)
12. Handling Loading, Errors & Validation (Deep Dive)
13. Global Error Handling (Error Boundaries + Axios Interceptor)
14. Pagination & Infinite Scroll
15. File Upload / Download Scenario
16. Real-Time Communication (WebSocket / STOMP over Spring Boot)
17. State Management Tied to API Calls (Context/Redux)
18. Environment Configuration (Dev/Prod, .env, Profiles)
19. API Versioning & Backward Compatibility
20. Testing the Integration (Jest, MSW, Spring MockMvc)
21. Security Considerations in Integration
22. Performance Optimization (Debouncing, Caching, Lazy Loading)
23. Deployment & CI/CD Considerations
24. Microservices & API Gateway with React
25. Real-World Scenario 1: E-commerce Checkout
26. Real-World Scenario 2: Dashboard with Live Data
27. Real-World Scenario 3: Multi-Step Form Wizard
28. Common Issues & Fixes (Extended)
29. Best Practices Checklist
30. Glossary of Terms

31. Interview Questions & Answers (30 Q&A)

32. Quick Revision Summary

1. How Frontend & Backend Talk (Big Picture)

React (frontend) and Spring Boot (backend) are two completely separate applications running on different ports/servers. They "talk" only through HTTP requests — the frontend never directly touches the backend's database or Java code.

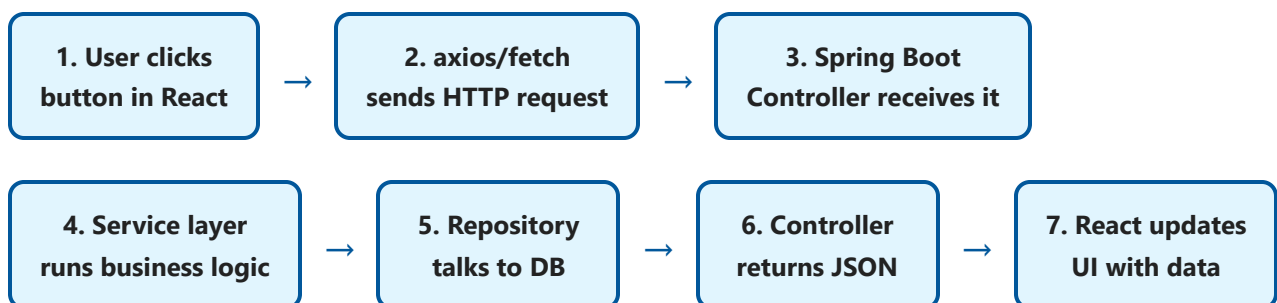


Real-time example: On an online food ordering site, when you click "Place Order", React sends a POST request with the cart JSON to `http://api.food.com/orders`. Spring Boot saves it to the database and sends back a JSON response with the order ID.

Two common deployment patterns

- **Separate deployments:** React built as static files served by Nginx/Vercel/Netlify; Spring Boot runs independently as an API-only service. Most common for modern apps.
- **Combined deployment:** React build output placed inside Spring Boot's `src/main/resources/static` folder, served by the same Spring Boot app — simpler ops, less flexible scaling.

2. Request-Response Flow (Visual)



Every one of these steps can fail independently — network drop, validation error, database timeout — which is why proper error handling on both sides is essential (explained in later sections).

3. REST API Basics

HTTP Method	Purpose	Example
GET	Read data	GET /api/products
POST	Create new data	POST /api/orders
PUT	Update entire record	PUT /api/products/5
PATCH	Update partial record	PATCH /api/products/5
DELETE	Remove data	DELETE /api/products/5

Status Code	Meaning
200 OK	Success
201 Created	New resource created successfully
204 No Content	Success, but nothing to return (common for DELETE)
400 Bad Request	Invalid input from client
401 Unauthorized	Not logged in / missing token
403 Forbidden	Logged in but not allowed
404 Not Found	Resource/URL doesn't exist
409 Conflict	Request conflicts with current state (e.g., duplicate entry)
422 Unprocessable Entity	Validation failed on otherwise well-formed request
429 Too Many Requests	Rate limit exceeded
500 Internal Server Error	Something broke on the backend

4. Spring Boot: Building the API (Layered Architecture)

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Autowired
    private ProductService service;

    @GetMapping
    public List<ProductDto> getAll() {
        return service.getAllProducts();
    }

    @PostMapping
    public ResponseEntity<ProductDto> create(@Valid @RequestBody ProductRequest request) {
        ProductDto saved = service.save(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }
}
```

```
@Service
public class ProductService {
    @Autowired private ProductRepository repository;
    @Autowired private ProductMapper mapper;

    public List<ProductDto> getAllProducts() {
        return repository.findAll().stream().map(mapper::toDto).toList();
    }

    public ProductDto save(ProductRequest request) {
        Product product = mapper.toEntity(request);
        return mapper.toDto(repository.save(product));
    }
}
```

Layers: **Controller** (handles HTTP) → **Service** (business logic) → **Repository** (database access, usually `JpaRepository`) → **Mapper** (converts between Entity and DTO so the API never leaks internal DB structure).

5. React Basics Recap (Components, State, Props, Hooks)

Before diving into API calls, a quick recap of the React concepts most relevant to backend integration:

Concept	Simple Meaning
Component	A reusable piece of UI (function that returns JSX).
Props	Data passed INTO a component from its parent (read-only).
State	Data owned BY a component that can change over time (<code>useState</code>).
<code>useEffect</code>	Runs side effects (like API calls) after render, or when dependencies change.
<code>useContext</code>	Shares data (like a logged-in user) across many components without prop-drilling.

```
function ProductPage() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchProducts().then(data => {
      setProducts(data);
      setLoading(false);
    });
  }, []); // empty array = run once when component mounts

  return loading ? <Spinner /> : <ProductList items={products} />;
}
```

6. React: Calling the API (fetch vs axios, Deep Dive)

Using fetch (built into the browser)

```
fetch("http://localhost:8080/api/products")
  .then(res => {
    if (!res.ok) throw new Error("Request failed: " + res.status);
    return res.json();
  })
  .then(data => setProducts(data))
  .catch(err => console.error(err));
```

Using axios (popular library)

```
import axios from "axios";

axios.get("http://localhost:8080/api/products")
  .then(res => setProducts(res.data))
  .catch(err => console.error("Error fetching products", err));
```

Feature	fetch	axios
JSON parsing	Manual (<code>res.json()</code>)	Automatic
Error on 4xx/5xx	No — you must check <code>res.ok</code> manually	Yes — throws automatically
Interceptors	Not built-in	Built-in (request/response interceptors)
Request cancellation	AbortController (manual)	CancelToken / AbortController support
Browser support	Native, no install needed	Requires npm install

POST request example

```
axios.post("http://localhost:8080/api/orders", {
  productId: 5,
  quantity: 2
}, {
  headers: { Authorization: `Bearer ${token}` }
});
```

7. Axios Interceptors & Centralized API Client

Instead of repeating headers/base URLs everywhere, create one shared axios instance.

```
// api/client.js
import axios from "axios";

const api = axios.create({
  baseURL: process.env.REACT_APP_API_URL,
  timeout: 10000
});

// Request interceptor - attach token automatically
api.interceptors.request.use(config => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Response interceptor - handle 401 globally
api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);

export default api;
```

Now every part of the app just imports `api` and calls `api.get("/products")` — token attachment and 401 handling happen automatically in one place.

8. Data Fetching Patterns (useEffect, React Query/SWR)

Plain useEffect (manual, more boilerplate)

```
useEffect(() => {  
  let cancelled = false;  
  setLoading(true);  
  api.get("/products")  
    .then(res => { if (!cancelled) setProducts(res.data); })  
    .catch(err => { if (!cancelled) setError(err); })  
    .finally(() => { if (!cancelled) setLoading(false); });  
  return () => { cancelled = true; }; // avoid state update after unmount  
}, []);
```

React Query (recommended for real apps)

```
import { useQuery } from "@tanstack/react-query";  
  
function ProductPage() {  
  const { data: products, isLoading, error } = useQuery({  
    queryKey: ["products"],  
    queryFn: () => api.get("/products").then(res => res.data)  
  });  
  
  if (isLoading) return <Spinner />;  
  if (error) return <ErrorMessage error={error} />;  
  return <ProductList items={products} />;  
}
```

React Query (or SWR) automatically handles caching, background refetching, retries, and loading/error states — removing a lot of manual `useEffect` boilerplate and reducing bugs like duplicate fetches or stale data.

9. CORS — Why It Breaks & How to Fix (Deep Dive)

Since React runs on `localhost:3000` and Spring Boot on `localhost:8080`, they are "different origins" from the browser's point of view. By default, browsers block such cross-origin calls for safety — this is CORS.

Preflight requests

For non-simple requests (e.g., using JSON body + custom headers like `Authorization`), the browser first sends an automatic `OPTIONS` request (a "preflight") to check if the real request is allowed, before sending the actual GET/POST.

ISSUE Console shows: "has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header".

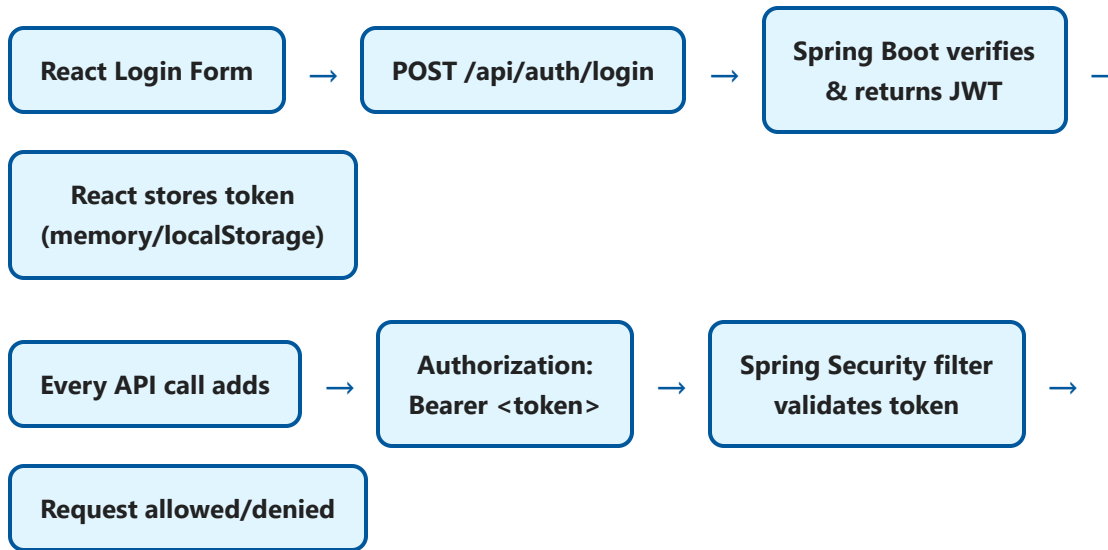
FIX Allow the frontend origin explicitly on the backend:

```
@CrossOrigin(origins = "http://localhost:3000")
@RestController
public class ProductController { ... }

// OR globally:
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:3000")
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```

If you use cookies/credentials, `allowedOrigins` cannot be `"*"` — you must specify the exact origin, and set `allowCredentials(true)` on both frontend (`withCredentials: true`) and backend.

10. Authentication Flow Between React & Spring Boot (JWT, Full Implementation)



React login page

```

function LoginPage() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const navigate = useNavigate();

  const handleLogin = async (e) => {
    e.preventDefault();
    try {
      const res = await api.post("/auth/login", { username, password });
      localStorage.setItem("token", res.data.token);
      navigate("/dashboard");
    } catch (err) {
      alert("Invalid username or password");
    }
  };

  return (
    <form onSubmit={handleLogin}>
      <input value={username} onChange={e => setUsername(e.target.value)} />
      <input type="password" value={password} onChange={e => setPassword(e.target.value)} />
      <button type="submit">Login</button>
    </form>
  );
}

```

Spring Boot side (matches the Spring Security JWT filter covered in the Security guide)

```
@PostMapping("/auth/login")
public ResponseEntity<AuthResponse> login(@RequestBody LoginRequest req) {
    authManager.authenticate(new UsernamePasswordAuthenticationToken(req.getUsername(),
    req.getPassword()));
    String token = jwtUtil.generateToken(req.getUsername());
    return ResponseEntity.ok(new AuthResponse(token));
}
```

11. Protecting React Routes (Private Routes)

```
function PrivateRoute({ children }) {
    const token = localStorage.getItem("token");
    return token ? children : <Navigate to="/login" />;
}

// usage in router
<Route path="/dashboard" element={
    <PrivateRoute><Dashboard /></PrivateRoute>
} />
```

This only hides the page in the UI — it does NOT replace backend security. The Spring Boot API must independently reject unauthorized requests, since a user could call the API directly (e.g., via Postman) bypassing the React route guard entirely.

12. Handling Loading, Errors & Validation (Deep Dive)

Good frontend-backend integration always handles 3 states: **loading**, **success**, and **error** — never assume the API call will always succeed instantly.

```
// Spring Boot validation error response
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String,String>> handleValidation(MethodArgumentNotValidException ex) {
    Map<String,String> errors = new HashMap<>();
    ex.getBindingResult().getFieldErrors()
        .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
    return ResponseEntity.badRequest().body(errors);
}
```

```
// React showing backend validation errors
const [fieldErrors, setFieldErrors] = useState({});

const submitForm = async (formData) => {
    try {
        await api.post("/products", formData);
    } catch (err) {
        if (err.response?.status === 400) {
            setFieldErrors(err.response.data); // { name: "must not be blank" }
        }
    }
};

// In JSX

```

13. Global Error Handling (Error Boundaries + Axios Interceptor)

```

class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(error, info) { console.error(error, info); }
  render() {
    if (this.state.hasError) return <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}

// Wrap the app
<ErrorBoundary>
  <App />
</ErrorBoundary>

```

Error Boundaries catch rendering errors (like a crash in a component), while axios interceptors (Section 7) catch network/API errors — together they cover both categories of failure a React app can hit.

14. Pagination & Infinite Scroll

```

// Spring Boot
@GetMapping
public Page<ProductDto> getProducts(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size) {
    return repository.findAll(PageRequest.of(page, size)).map(mapper::toDto);
}

```

```

// React
const [page, setPage] = useState(0);
const { data } = useQuery({
  queryKey: ["products", page],
  queryFn: () => api.get(`/products?page=${page}&size=20`).then(r => r.data)
});

<button onClick={() => setPage(p => p - 1)} disabled={page === 0}>Prev</button>
<button onClick={() => setPage(p => p + 1)}>Next</button>

```

Real-time example: An infinite-scrolling product feed calls `/api/products?page=N&size=20` each time the user scrolls near the bottom, appending new results to the existing list rather than replacing it.

15. File Upload / Download Scenario

```
// React - upload
const formData = new FormData();
formData.append("file", selectedFile);
api.post("/upload", formData, {
  headers: { "Content-Type": "multipart/form-data" }
});

// Spring Boot - receive
@PostMapping("/upload")
public ResponseEntity<String> upload(@RequestParam("file") MultipartFile file) {
  // save file.getBytes() to disk/S3
  return ResponseEntity.ok("Uploaded: " + file.getOriginalFilename());
}
```

```
// React - download a file (e.g., PDF invoice)
const downloadInvoice = async (orderId) => {
  const res = await api.get(`/orders/${orderId}/invoice`, { responseType: "blob" });
  const url = window.URL.createObjectURL(new Blob([res.data]));
  const link = document.createElement("a");
  link.href = url;
  link.setAttribute("download", `invoice-${orderId}.pdf`);
  document.body.appendChild(link);
  link.click();
};
```


16. Real-Time Communication (WebSocket / STOMP over Spring Boot)

REST is request-response only — the server can't "push" updates. For live features (chat, notifications, live order tracking), use WebSockets.

```
// Spring Boot config
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws").setAllowedOrigins("http://localhost:3000").withSockJS();
    }
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/topic");
        registry.setApplicationDestinationPrefixes("/app");
    }
}

@Controller
public class OrderStatusController {
    @MessageMapping("/order-status")
    @SendTo("/topic/order-updates")
    public OrderStatus broadcast(OrderStatus status) {
        return status;
    }
}
```

```
// React client (using @stomp/stompjs + sockjs-client)
const client = new Client({
    websocketFactory: () => new SockJS("http://localhost:8080/ws"),
    onConnect: () => {
        client.subscribe("/topic/order-updates", (msg) => {
            const update = JSON.parse(msg.body);
            setOrderStatus(update.status);
        });
    }
});
client.activate();
```

Real-time example: A food delivery app shows "Your rider is 5 minutes away" updating live without refreshing the page — the backend pushes updates over WebSocket as soon as the rider's location event arrives from Kafka/GPS tracking.

17. State Management Tied to API Calls (Context/Redux)

For data needed across many components (logged-in user, cart items), a global store avoids "prop drilling" (passing props through many layers).

```
// Simple Context example
const AuthContext = createContext();

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = async (credentials) => {
    const res = await api.post("/auth/login", credentials);
    localStorage.setItem("token", res.data.token);
    setUser(res.data.user);
  };

  return (
    <AuthContext.Provider value={{ user, login }}>
      {children}
    </AuthContext.Provider>
  );
}

// any component
const { user, login } = useContext(AuthContext);
```

Use Context/Redux for cross-cutting state like auth/user/cart. For pure "server data" (lists, details), prefer React Query/SWR instead of duplicating it into Redux — it already handles caching and avoids keeping two sources of truth in sync.

18. Environment Configuration (Dev/Prod, .env, Profiles)

```
# React: .env.development
REACT_APP_API_URL=http://localhost:8080/api

# React: .env.production
REACT_APP_API_URL=https://api.myproductionapp.com/api
```

```
# Spring Boot: application-dev.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/devdb

# Spring Boot: application-prod.yml
spring:
  datasource:
    url: jdbc:mysql://prod-db-host:3306/proddb
```

```
java -jar app.jar --spring.profiles.active=prod
```

Never hardcode `localhost:8080` directly in React components — always read from the environment variable so the same build process works across dev/staging/prod.

19. API Versioning & Backward Compatibility

Strategy	Example
URI versioning	<code>/api/v1/products</code> , <code>/api/v2/products</code>
Header versioning	<code>Accept: application/vnd.company.v2+json</code>
Query param versioning	<code>/api/products?version=2</code>

URI versioning is the most common and simplest for teams working with a React frontend, since it's visible and easy to route/proxy.

20. Testing the Integration

Spring Boot side — MockMvc

```
@WebMvcTest(ProductController.class)
class ProductControllerTest {
    @Autowired MockMvc mockMvc;
    @MockBean ProductService service;

    @Test
    void shouldReturnProducts() throws Exception {
        when(service.getAllProducts()).thenReturn(List.of(new ProductDto(1L, "Phone")));
        mockMvc.perform(get("/api/products"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[0].name").value("Phone"));
    }
}
```

React side — Jest + Mock Service Worker (MSW)

```
// mocks/handlers.js
export const handlers = [
    rest.get("/api/products", (req, res, ctx) =>
        res(ctx.json([{ id: 1, name: "Phone" }]))
    )
];

// ProductPage.test.js
test("renders product list from API", async () => {
    render(<ProductPage />);
    expect(await screen.findByText("Phone")).toBeInTheDocument();
});
```

MSW intercepts network calls during tests so you can verify component behavior without hitting a real backend — fast, reliable, and independent of backend uptime.

21. Security Considerations in Integration

- Never trust frontend-calculated values (price, discounts) — always re-validate server-side.
- Sanitize/escape any user-generated content rendered in React to avoid XSS.
- Store JWTs carefully — prefer memory or HttpOnly cookies over localStorage for high-security apps.
- Always use HTTPS in production for both the React app and the Spring Boot API.
- Return generic error messages to the client (avoid leaking stack traces or internal details).

ISSUE Detailed Java stack trace shown in the browser console/network tab on a 500 error.

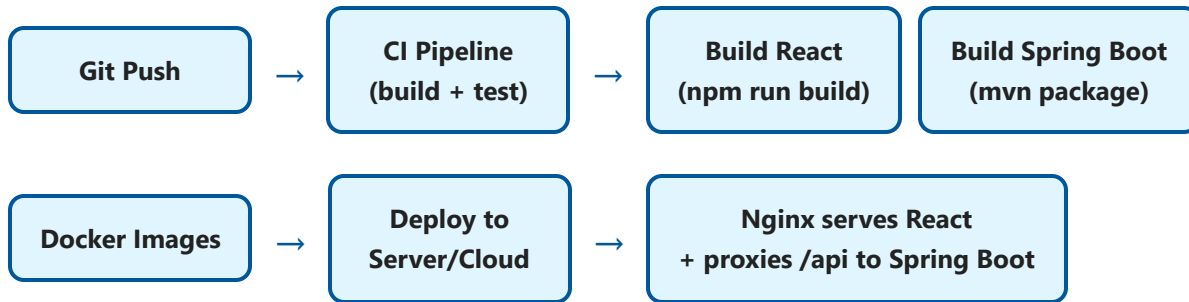
FIX Add a global `@ControllerAdvice` exception handler that returns a clean, generic JSON error instead of the default Whitelabel error page/stack trace.

22. Performance Optimization

- **Debounce** search inputs so React doesn't call the API on every keystroke.
- **Cache** GET responses with React Query/SWR to avoid redundant network calls.
- **Lazy-load** React routes/components (`React.Lazy`) to reduce initial bundle size.
- On the backend, add **pagination** and avoid returning huge unbounded lists.
- Use HTTP **compression** (gzip) on Spring Boot responses for large JSON payloads.

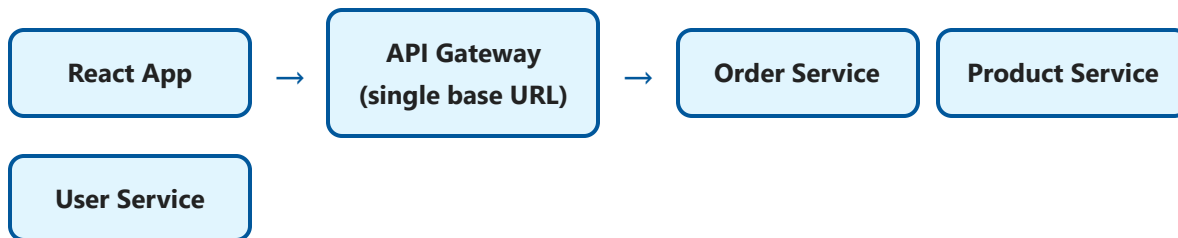
```
// Debounce example
const debouncedSearch = useMemo(
  () => debounce((term) => api.get(`/products/search?q=${term}`).then(...), 400),
  []
);
```

23. Deployment & CI/CD Considerations



```
# Example nginx.conf snippet
location /api/ {
    proxy_pass http://backend:8080/api/;
}
location / {
    root /usr/share/nginx/html; # React build output
    try_files $uri /index.html;
}
```

24. Microservices & API Gateway with React



React only ever talks to ONE base URL (the gateway); the gateway routes each request to the correct backend microservice, hiding internal architecture changes from the frontend completely.

25. Real-World Scenario 1: E-commerce Checkout

1. React loads product list via `GET /api/products` on page load (cached with React Query).
2. User clicks "Add to Cart" → stored in React Context/local state (client side, no API call yet).
3. User clicks "Checkout" → React sends `POST /api/orders` with cart items + JWT token.
4. Spring Boot validates stock, re-calculates price server-side, saves order, calls Payment Service, returns order confirmation JSON.
5. React shows a success page with the order ID, or an error message with retry option if payment failed.

Key point: Frontend never trusts its own price calculation for payment — the backend always re-validates price/stock server-side to prevent tampering (e.g., user changing price in browser dev tools).

26. Real-World Scenario 2: Dashboard with Live Data

An admin dashboard shows live order counts and revenue, refreshing automatically.

- Initial load via `GET /api/dashboard/summary`.
- React Query configured with `refetchInterval: 5000` to poll every 5 seconds, OR a WebSocket subscription for true push-based updates.
- Charts re-render only when new data actually differs, avoiding unnecessary flicker.

27. Real-World Scenario 3: Multi-Step Form Wizard

A loan application form spans 4 steps (personal info, income, documents, review).

- Each step's data is saved in React state and only sent to the backend as a `PATCH /api/applications/{id}` when the user clicks "Next", allowing resuming later.
- Final submit calls `POST /api/applications/{id}/submit`, which the backend validates as a whole before finalizing.
- Validation errors from Spring Boot are mapped back to the specific step/field so the user isn't confused about where the problem is.

28. Common Issues & Fixes (Extended)

ISSUE CORS error blocking all API calls from React.

FIX Configure `@CrossOrigin` or a global CORS bean on Spring Boot allowing the React origin.

ISSUE "Network Error" in axios with no response.

FIX Backend not running / wrong port / wrong base URL in React `.env` file.

ISSUE 401 Unauthorized on every API call after login worked fine.

FIX Token not attached to request headers, or token expired — check axios interceptor.

ISSUE React shows stale/old data after update.

FIX Re-fetch data after a successful POST/PUT, invalidate the React Query cache key, or update local state directly with the response data.

ISSUE Date/number format mismatch between React and Spring Boot.

FIX Standardize on ISO 8601 dates (`yyyy-MM-dd'T'HH:mm:ss`) and always send/receive JSON, not locale-formatted strings.

ISSUE Large payload / slow API makes UI freeze or feel unresponsive.

FIX Add pagination (`?page=0&size=20`) on backend, show loading spinners, debounce search inputs on frontend.

ISSUE Sensitive data (password, internal IDs) accidentally returned in API response.

FIX Use DTOs instead of returning JPA entities directly; annotate fields with `@JsonIgnore` where needed.

ISSUE Duplicate order/form submitted when user double-clicks submit button.

FIX Disable the button after first click on the frontend, and use idempotency keys on backend for critical operations.

ISSUE WebSocket connection keeps disconnecting/reconnecting.

FIX Check reverse proxy (Nginx) timeout settings and ensure it's configured to support WebSocket upgrade headers.

ISSUE File upload fails with 413 "Payload Too Large".

FIX Increase `spring.servlet.multipart.max-file-size / max-request-size` and any reverse proxy body size limits.

29. Best Practices Checklist

- Use one centralized axios instance with interceptors instead of scattering headers everywhere.
- Always show loading/success/error UI states — never leave the user guessing.
- Validate on both frontend (fast feedback) AND backend (source of truth/security).
- Use DTOs on the backend; never expose raw JPA entities.
- Store secrets/base URLs in environment variables, not hardcoded in components.
- Paginate large lists; debounce search inputs.
- Write integration tests on both sides (MockMvc for Spring Boot, MSW/Jest for React).
- Use HTTPS everywhere in production; never send JWTs over plain HTTP.

30. Glossary of Terms

Term	Meaning
DTO	Data Transfer Object — a plain object used to shape API request/response data.
CORS Preflight	An automatic OPTIONS request the browser sends before certain cross-origin requests.
Interceptor	A function that runs before a request is sent or after a response is received (axios feature).
SPA	Single Page Application — a web app that loads once and updates content dynamically via JS.
Idempotency Key	A unique key sent with a request so retrying it doesn't create duplicate side effects.
API Gateway	A single entry point that routes client requests to the correct backend microservice.

31. Interview Questions & Answers (30 Q&A)

Q1. How does React communicate with a Spring Boot backend?

A: Through HTTP REST calls (using fetch or axios) — React sends JSON requests to Spring Boot REST endpoints and receives JSON responses, which it uses to update the UI.

Q2. What is CORS and why does it happen between React and Spring Boot?

A: Cross-Origin Resource Sharing — a browser security rule blocking requests between different origins (different port/domain). It happens because React (3000) and Spring Boot (8080) are considered different origins; fixed by enabling CORS on the backend.

Q3. How do you send an auth token with every React API request?

A: Using an axios interceptor that automatically attaches `Authorization: Bearer <token>` to every outgoing request.

Q4. Why use DTOs instead of returning entity objects directly from Spring Boot?

A: To control exactly what fields are exposed to the frontend, avoid exposing sensitive data, and avoid issues like circular references / lazy-loading exceptions from JPA entities.

Q5. How do you handle validation errors between frontend and backend?

A: Backend uses `@Valid` + `@ExceptionHandler` to return structured error JSON (400 status with field-level messages); frontend catches this in the axios error response and displays it near the relevant form field.

Q6. What is the difference between fetch and axios?

A: Both make HTTP calls; axios automatically parses JSON, supports interceptors, and throws on non-2xx responses automatically, while fetch is a native browser API requiring manual JSON parsing and manual status checks.

Q7. How would you implement pagination between React and Spring Boot?

A: Backend uses Spring Data's `Pageable` (e.g., `GET /api/products?page=0&size=10`) returning a `Page<Product>` object; React keeps a page number in state and refetches when the user clicks Next/Previous.

Q8. Why should sensitive business logic (like price calculation) never be trusted from the frontend?

A: Frontend code and network requests can be modified by users (browser dev tools, Postman); the backend must always re-validate and recalculate critical values server-side to prevent tampering/fraud.

Q9. How do you handle file uploads from React to Spring Boot?

A: React sends a `multipart/form-data` request using `FormData`; Spring Boot receives it via `@RequestParam("file") MultipartFile file`.

Q10. What HTTP status code should a successful POST that creates a resource return?

A: 201 Created, ideally along with a `Location` header pointing to the new resource.

Q11. How do you keep the frontend UI in sync after updating data on the backend?

A: Either re-fetch the list after a successful mutation, invalidate the relevant React Query cache key, or optimistically update local state with the response returned from the POST/PUT call.

Q12. What's a good way to manage different API base URLs for dev/prod in React?

A: Use environment variables (e.g., `REACT_APP_API_URL` in a `.env` file) instead of hardcoding `localhost:8080` in the code.

Q13. How do you prevent duplicate form submissions in a React + Spring Boot app?

A: Disable the submit button immediately on click (frontend) and optionally use an idempotency key or unique constraint check on the backend for critical operations like payments.

Q14. How does a Single Page Application (SPA) handle a 401 response globally?

A: Using an axios response interceptor that detects 401 status, clears the stored token, and redirects the user to the login page automatically.

Q15. What is the role of an API Gateway in a React + multiple Spring Boot microservices setup?

A: It acts as a single entry point that routes React's requests to the correct microservice, and can centrally handle concerns like authentication, rate limiting, and CORS.

Q16. What is a CORS preflight request?

A: An automatic `OPTIONS` request the browser sends before certain cross-origin requests (e.g., with custom headers or non-simple methods) to check if the server permits the actual request.

Q17. How would you implement real-time updates from Spring Boot to React?

A: Use WebSockets (with STOMP over SockJS in Spring Boot) so the server can push updates to connected React clients instantly, instead of the client repeatedly polling the API.

Q18. What is the benefit of using React Query/SWR over plain `useEffect` for API calls?

A: They provide built-in caching, automatic background refetching, request deduplication, and loading/error state management, reducing boilerplate and common bugs.

Q19. How do you protect a React route so only logged-in users can access it?

A: Create a wrapper component (e.g., `PrivateRoute`) that checks for a valid token/user in state and redirects to login if missing — but this must always be backed by real server-side authorization, since frontend route guards are not real security.

Q20. How do you debounce a search input calling a Spring Boot search API?

A: Wrap the API call in a debounce function (e.g., via lodash or a custom hook) so it only fires after the user stops typing for a short delay, reducing unnecessary requests.

Q21. Why is API versioning important, and how is it typically implemented?

A: It lets you evolve the backend without breaking existing frontend clients; most commonly implemented via URI versioning like `/api/v1/...` and `/api/v2/...`.

Q22. How would you test that a React component correctly displays data from an API?

A: Use Mock Service Worker (MSW) to intercept the API call during a Jest/React Testing Library test, returning mock data, then assert the expected text/elements render.

Q23. What's the difference between optimistic and pessimistic UI updates?

A: Optimistic updates change the UI immediately assuming success (and roll back on error); pessimistic updates wait for the server's confirmed response before updating the UI — optimistic feels faster but requires careful rollback handling.

Q24. How do you handle large file downloads (like PDF invoices) in React?

A: Request with `responseType: 'blob'` in axios, then create an object URL from the blob and trigger a download via a temporary anchor element.

Q25. Why should you avoid storing sensitive tokens in plain localStorage?

A: localStorage is accessible to any JavaScript running on the page, making tokens vulnerable to XSS attacks; HttpOnly cookies are safer against script-based theft but need CSRF protection instead.

Q26. How does a reverse proxy like Nginx fit into a React + Spring Boot deployment?

A: It serves the built React static files and proxies API requests (e.g., `/api/**`) to the Spring Boot backend, letting both be accessed under a single domain without CORS issues.

Q27. What causes a "stale closure" bug when calling APIs inside useEffect?

A: Referencing outdated state/props inside an effect due to missing dependencies in the dependency array — fixed by including all used variables in the dependency array or using functional state updates.

Q28. How do you avoid exposing internal error details (stack traces) to the React frontend?

A: Implement a global `@ControllerAdvice` exception handler in Spring Boot that catches exceptions and returns a clean, generic error message/JSON instead of the default error page.

Q29. What is the purpose of a loading skeleton/spinner in React during API calls?

A: It gives immediate visual feedback that data is being fetched, improving perceived performance and

preventing users from thinking the app is frozen or broken.

Q30. How would you design pagination for an infinite-scroll product feed?

A: Backend exposes page-based or cursor-based pagination; React tracks the current page/cursor, fetches the next batch when the user scrolls near the bottom, and appends results to the existing list rather than replacing it.

32. Quick Revision Summary

- React and Spring Boot communicate only via HTTP REST calls using JSON (or WebSockets for real-time push).
- Always handle loading/success/error states on the frontend.
- CORS must be explicitly enabled on the backend for cross-origin requests; preflight requests happen automatically for non-simple calls.
- Use a centralized axios instance with interceptors for tokens and global 401 handling.
- Send JWT via `Authorization: Bearer` header for protected APIs; private React routes are UX only, not real security.
- Use DTOs, not raw entities, to control API responses.
- Never trust frontend data for critical logic — backend must re-validate.
- Use pagination for large lists, debounce search inputs, and disable buttons to avoid duplicate submits.
- Test both sides independently: MockMvc (Spring Boot), MSW/Jest (React).